
基于激光雷达的移动机器人建图分析案例

目录

基于激光雷达的移动机器人建图分析案例	1
1.1 地图的表现形式	1
1.2 移动机器人模型	3
1.2.1 机器人坐标系统	3
1.2.2 机器人运动模型	5
1.3 常用激光 SLAM 算法介绍	7
1.1.1 Cartographer 算法	7
1.1.2 Hector SLAM 算法	8
1.1.2 Gmapping 算法	10
1.4 SLAM 建图仿真	13
1.4.1 仿真环境的搭建	14
1.4.2 建图质量评价方法	15
1.4.3 建图关键参数的选择	17
1.5 实验与分析	20

机器人在未知环境下建图，可以根据传感器的不同分为激光和视觉建图。视觉传感器可以获取丰富的环境信息，在室内环境实现较高精度的定位，但是视觉传感器易受光照以及机器人运动的干扰，使用激光雷达的建图方法在系统稳定性上更有优势。另一方面，使用激光传感器建立的栅格地图可以很好地描述障碍之间的关系，基于栅格图的导航算法也层出不穷。本节使用激光雷达仿真方舱环境下的建图，并总结出关键参数的选值规律。

1.1 地图的表现形式

(1) 几何特征地图

机器人在探索时，未知的环境特征通常可由各种规则的多边形或者曲线进行拟合表示。使用这种环境特征的表示方法优点在于形象，直观，并且地图的存储消耗较低。该环境建模方法在室内结构化环境下可以建立较高精度的地图。但是当应用在室外环境时，例如森林环境，大型商场环境等非结构化环境，该方法很难准确地通过简单的几何图形的组合描述环境信息，建立的地图精度差。特征地图如图 1.1 所示。

一个栅格代表现实环境 $5\text{cm} \times 5\text{cm}$ 的区域。如果对应栅格的真实环境由障碍物，则将该栅格标记为黑色，表示障碍栅格；如果对应栅格的真实环境不存在障碍物，则将该栅格标记为白色，表示自由栅格。栅格地图的优点在于简便、直观，本文建图与导航都是基于栅格地图。栅格地图如图 1.3 所示。

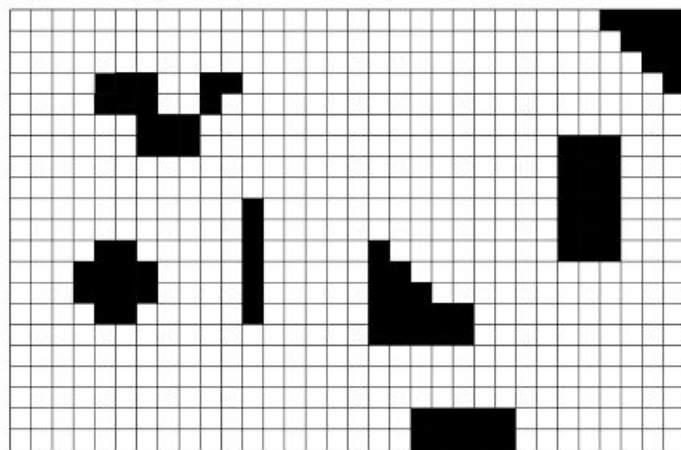


图 1.3 栅格地图

Fig 1.3 Raster map

1.2 移动机器人模型

1.2.1 机器人坐标系统

坐标系统是机器人建图与定位中重要的组成部分，它主要用来表示机器人各个模块的位姿变换关系。通过坐标系统，机器人可以将传感器观测数据以及机器人本身的运动数据联系起来实现对环境的感知。不同坐标系之间的转换关系可以平移量 T 和旋转量 R 进行表示，其中 T 包括三个分量： T_x, T_y, T_z ；旋转量 R 也可以用三个分量表示：偏航角 yaw ，俯仰角 $pitch$ ，翻滚角 $roll$ 。以 Gmapping 建图算法为例，机器人主要坐标系统如图 1.4 所示。

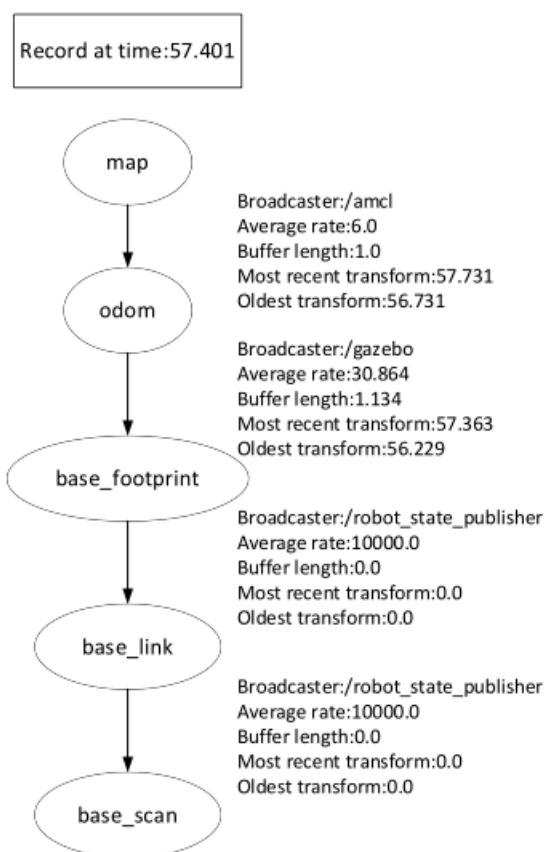


图 1.4 Gmapping 算法机器人坐标系统

Fig 1.4 Robot coordinate system of Gmapping algorithm

上图列出了五个主要的坐标系，上面的坐标系是下面坐标系的父坐标系，具有直接的转换关系。/base_laser 代表的是激光雷达坐标系，激光雷达运行时扫描得到的数据都是基于此坐标系；/base_link 代表的是机器人底盘坐标系，一般表示机器人中心的坐标系。/base_footprint 为机器人坐标系在地面的投影坐标系。/odom 是里程计坐标系，/map 是地图坐标系，一般情况下与初始状态的/base_link 坐标系重合。/map 与/odom 之间的关系反映了机器人里程计的误差，理想情况下两者重合。当机器人建图出现意外错误时，理解坐标系转换关系有助于排查问题。

本文使用激光雷达构建栅格地图，因此需要给定/base_link 到/base_laser 之间的坐标转换关系。当激光雷达实际放置位置与机器人中心位置关系难以直接测量得出时，需要做激光雷达外参标定。本系统激光雷达平放在机器人顶层支架，且激光雷达坐标系与机器人坐标系旋转关系为 180 度，所以两者坐标转换关系为：0, 0, d, 1.14, 0, 0。示意如图 1.5 所示。

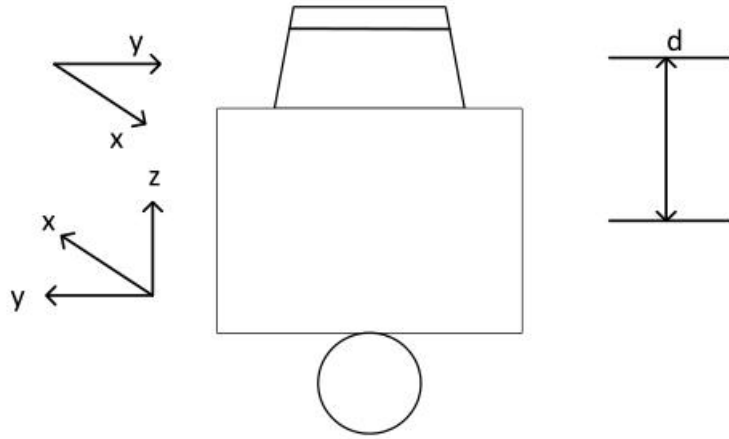


图 1.5 激光雷达与机器人坐标系关系

Fig 1.5 Coordinate relation between lidar and robot

1.2.2 机器人运动模型

建立移动机器人运动学模型时后续建图和定位的基础，本系统使用的机器人基于两轮差速模型，差速轮结构简单，控制方便，不受室内外环境限制，因而广泛使用。机器人差速轮模型如图 1.6 所示。

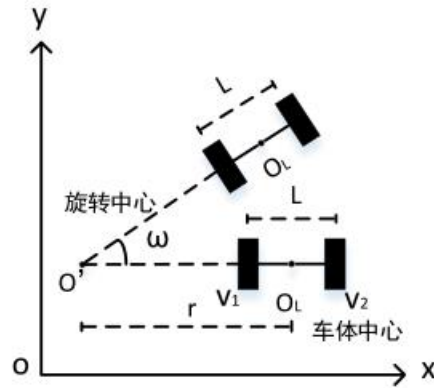


图 1.6 机器人差速轮模型

Fig 1.6 Robot differential wheel model

机器人的运动过程可以用离散圆弧运动近似描述， O' 为圆弧运动的中心， O_L 为机器人车体中心， r 代表圆弧运动的半径， L 代表机器人底盘直径，左轮和右轮的运动角速度和线速度分别 ω_l 、 ω_r 和 v_l 、 v_r 。运动过程中，机器人整体的角速度为 ω ，则根据运动学关系可得：

$$\omega = \frac{v_l}{r - L/2} = \frac{v_r}{r - L/2} \quad (1.1)$$

$$v_l(r + L/2) = v_r(r - L/2) \quad (1.2)$$

$$(v_r - v_l)r = (v_r - v_l)L/2 \quad (1.3)$$

$$r = \frac{(v_r + v_l) L}{(v_r - v_l) 2} \quad (1.4)$$

由式(1.4)可以看出,当机器人左右轮速度固定且不一致时,机器人做半径为 r 的圆周运动,而当左右轮速度一样大时,机器人圆周运动的半径为无穷大,此时机器人直线运动。

进一步推导可得:

$$r + \frac{L}{2} = \frac{(v_r + v_l) L}{(v_r - v_l) 2} + \frac{(v_r - v_l) L}{(v_r - v_l) 2} = \frac{v_r L}{v_r - v_l} \quad (1.5)$$

代入式(1.1)可得:

$$\omega = \frac{v_r - v_l}{L} \quad (1.6)$$

根据角速度线速度关系得:

$$v = \omega r = \frac{v_r - v_l}{L} \frac{(v_r + v_l) L}{(v_r - v_l) 2} = \frac{v_r + v_l}{2} \quad (1.7)$$

由上面的公式推导可以总结: 差速轮小车运动时,可以根据左右轮速度信息求出机器人运动的整体线速度与角速度。Kobiki 底盘配有光电编码器,实际运行时,机器人轮子的速度可以由编码器测量获得。假定机器人在运动过程中,轮子与地面时刻保持接触且不考虑打滑的影响,则根据以上条件,建立机器人运动学模型,如图 1.7 所示。

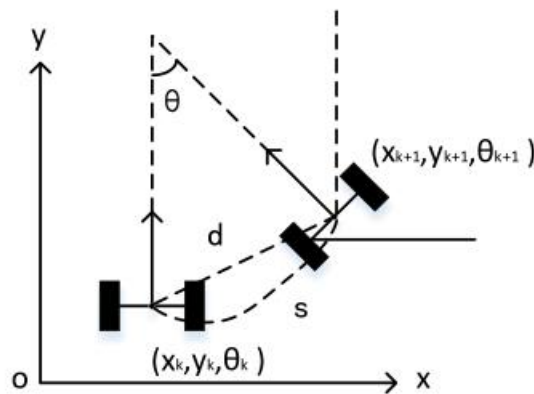


图 1.7 机器人运动学模型

Fig 1.7 Kinematics model of robot

令机器人 k 时刻的位姿为 (x_k, y_k, θ_k) , 机器人的运动速度为 (v, ω) , θ 表示机器人圆弧运动转过的角度, s 表示相邻时间段内机器人走过的弧线距离, d 表示

相邻时间段机器人的直线距离，当机器人移动前后时间段极小时有 $d \approx s$ ，则：

$$\begin{cases} \theta = \omega \Delta t \\ d \approx s = v \Delta t \end{cases} \quad (1.8)$$

$k+1$ 时刻机器人的位姿表示为 $(x_{k+1}, y_{k+1}, \theta_{k+1})$ ，由几何关系可得：

$$\begin{cases} x_{k+1} = x_k + d \cos(\theta_k + \frac{\theta}{2}) \\ y_{k+1} = y_k + d \sin(\theta_k + \frac{\theta}{2}) \\ \theta_{k+1} = \theta_k + \theta \end{cases} \quad (1.9)$$

将式(1.8)代入到上式可得：

$$\begin{cases} x_{k+1} = x_k + v \Delta t \cos(\theta_k + \frac{\omega \Delta t}{2}) \\ y_{k+1} = y_k + v \Delta t \sin(\theta_k + \frac{\omega \Delta t}{2}) \\ \theta_{k+1} = \theta_k + \omega \Delta t \end{cases} \quad (1.10)$$

根据以上推到的公式，机器人在得知自身位姿以及运动角速度和线速度信息后即可估计出下一时刻自身的位姿信息，这就是轮式机器人的运动学模型。

1.3 常用激光 SLAM 算法介绍

移动机器人在未知环境中自主导航完成任务的前提是具备自我定位、环境感知的能力。同时定位与建图（SLAM）决定着是否能够实现上述功能，建图的精度决定着后续导航的准确性，针对特定的环境选择一个合适的 SLAM 算法是很重要的，本小节主要介绍三种常用的 SLAM 算法原理，并选择合适的算法进行后续的研究。

1.1.1 Cartographer 算法

Cartographer 算法是谷歌在 2016 年提出的开源建图算法^[27]，它基于图优化的方法对现实环境建立二维或者三维的地图。相比起其它建图算法，该算法可以建立高精度的地图。Cartographer 主要依赖激光雷达建图，后续已经可以支持更多硬件。算法可以构建分辨率为 5cm 的栅格地图，并能够实时检测是否经过闭环从而减少累积误差。

Cartographer 算法框架如图 1.8 所示。框架的左侧是输入端，输入的信息包括激光雷达数据，里程计数据以及 IMU 数据。雷达的数据要经过两次滤波以减少数据量，包括一个边长固定的体素滤波以及一个自适应的体素滤波。IMU 的

数据通过 Imutracker 模块计算重力加速度并和里程计数据融合，实现对机器人位姿的初步估计，之后使用 Scan Matching 模块对位姿进一步优化。Extrapolator 模块保留一段时间机器人的位姿以计算线速度和角速度，同时利用该数据对激光雷达的数据畸变进行矫正。

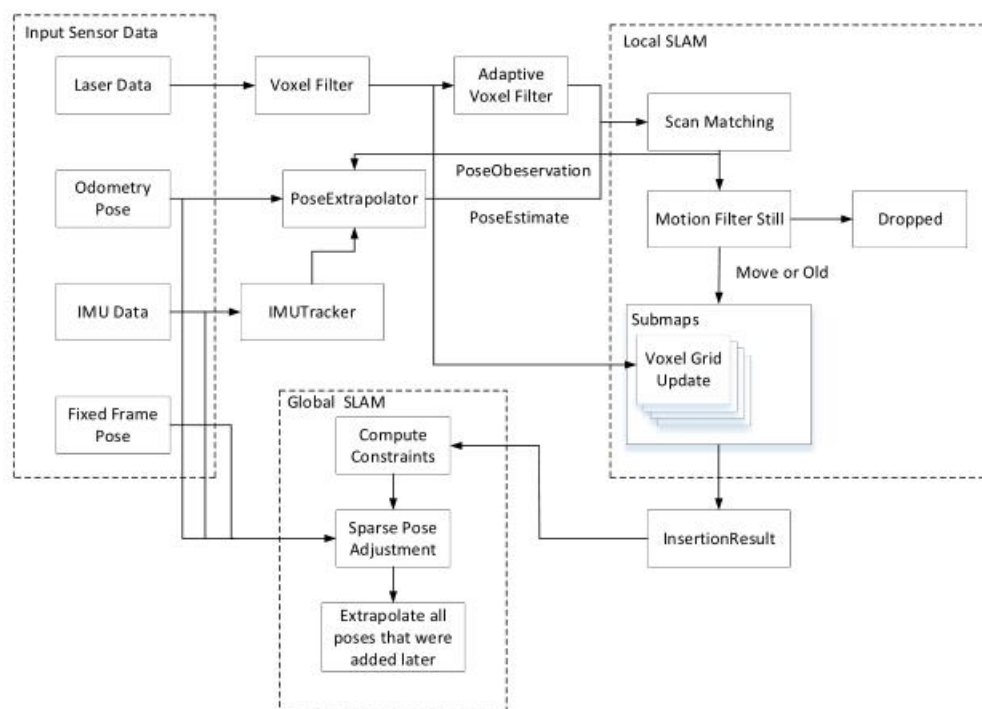


图 1.8 Cartographer 算法结构图

Fig 1.8 Map of the Cartographer algorithm

结构图的右方是 Local SLAM 部分，接收到激光雷达的数据后，模块调用 ceres 非线性优化库进行 Scan Matching 求解。每当求解到一个有效位姿后，模块将当前帧的激光点插入到子图中，如果计算出来机器人没有运动或者相对静止，则将数据删除处理下一帧。对激光数据逐帧处理后，算法会不断更新局部地图，最后得到一个完整的子图。

结构图的下方是 Global SLAM 部分，该模块的功能主要是进行后端的回环检测，每当机器人检测到经过同一个位置时，则加入约束对各个子图状态进行调整。为了加快计算速度，算法采用了分定支界的方法。

1.1.2 Hector SLAM 算法

Hector SLAM 算法将激光当前帧数据与先验地图匹配从而增量式构建环境地图^[28]。与其他激光建图算法相比，该方法不需要里程计辅助定位，因而主要被应用在地面不平坦的环境中。Hector SLAM 分为两个阶段，包括地图获取和扫描匹配，定位和建图两个部分相互循环促进。在每个周期中，环境信息被离散化并与前一时刻的地图进行匹配实现位姿估计，更新局部地图。

在地图获取阶段，建立地图的离散性不方便求导，可以将栅格节点之间看作连续分布并使用双线性法进行插值，根据已知点 P_{00} 、 P_{01} 、 P_{10} 和 P_{11} ，可以插值得出 P_m 的近似值，如图 1.9 所示。

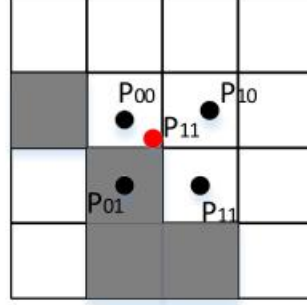


图 1.9 双线性滤波示意图

Fig 1.9 Schematic diagram of bilinear filtering

在 x 轴和 y 轴进行双线性滤波，可得：

$$M(P_m) \approx \frac{y-y_0}{y_1-y_0} \left[\frac{x-x_0}{x_1-x_0} M(P_{11}) + \frac{x_1-x}{x_1-x_0} M(P_{01}) \right] + \frac{y_1-y}{y_1-y_0} \left[\frac{x-x_0}{x_1-x_0} M(P_{10}) + \frac{x_1-x}{x_1-x_0} M(P_{00}) \right] \quad (1.11)$$

将上式中 $M(P_m)$ 分别对 x 轴和 y 轴求偏导可得下列两式，从而获取地图。

$$\frac{\partial M(P_m)}{\partial P_m} = \left[\frac{\partial M(P_m)}{\partial x}, \frac{\partial M(P_m)}{\partial y} \right] \quad (1.12)$$

$$\begin{cases} \frac{\partial M}{\partial x}(P_m) \approx \frac{y-y_0}{y_1-y_0} [M(P_{11}) - M(P_{01})] + \frac{y_1-y}{y_1-y_0} [M(P_{10}) - M(P_{00})] \\ \frac{\partial M}{\partial y}(P_m) \approx \frac{x-x_0}{x_1-x_0} [M(P_{11}) - M(P_{10})] + \frac{x_1-x}{x_1-x_0} [M(P_{01}) - M(P_{00})] \end{cases} \quad (1.13)$$

在扫描匹配阶段，算法用第一帧的激光雷达数据构建起始地图，后续的激光数据则与已构建的地图进行匹配，并计算出当前机器人的最优位姿

$x_t^* = (p_x, p_y, \psi)$ ，具体的推导过程如下：

$$x_t^* = \arg \min_{x_t} \sum_{i=1}^n [1 - M(S_i(x_t))]^2 \quad (1.14)$$

式中 $S_i(x_t)$ 表示传感器扫描端点 $s_i = (s_{i,x}, s_{i,y})^T$ 在全局坐标系下的坐标，转换过程如式(1.15)所示。 $M(S_i(x_t))$ 表示坐标为 $S_i(x_t)$ 的栅格的取值，采用高斯-牛顿梯度法的方法，设定一个初始值 x_t 估计 Δx_t 。

$$S_i(x_t) = \begin{bmatrix} \cos(\psi) & -\sin(\psi) \\ \sin(\psi) & \cos(\psi) \end{bmatrix} \begin{bmatrix} s_{i,x} \\ s_{i,y} \end{bmatrix} + \begin{bmatrix} p_x \\ p_y \end{bmatrix} \quad (1.15)$$

$$\sum_{i=1}^n [1 - M(S_i(x_t + \Delta x_t))]^2 \rightarrow 0 \quad (1.16)$$

将 $M(S_i(x_t + \Delta x_t))$ 一阶泰勒展开，对 Δx_t 求偏导，过程如下：

$$\begin{aligned} & \nabla_{x_t} \sum_{i=1}^n [1 - M(S_i(x_t + \Delta x_t))]^2 \\ & \approx \sum_{i=1}^n [1 - M(S_i(x_t)) - \nabla M(S_i(x_t)) \frac{\partial S_i(x_t)}{\partial x_t} \Delta x_t]^2 \\ & = 2 \sum_{i=1}^n [\nabla M(S_i(x_t)) \frac{\partial S_i(x_t)}{\partial x_t}]^T [1 - M(S_i(x_t)) - \nabla M(S_i(x_t)) \frac{\partial S_i(x_t)}{\partial x_t} \Delta x_t] \end{aligned} \quad (1.17)$$

令其结果为零，可以推出式子的解：

$$\Delta x_t = H^{-1} \sum_{i=1}^n [\nabla M(S_i(x_t)) \frac{\partial S_i(x_t)}{\partial x_t}]^T [1 - M(S_i(x_t))] \quad (1.18)$$

在上式中， $\nabla M(P_m)$ 代表在全局地图中点 P_m 的梯度， H 为海森矩阵，如式所示。

$$H = [\nabla M(S_i(x_t)) \frac{\partial S_i(x_t)}{\partial x_t}]^T [\nabla M(S_i(x_t)) \frac{\partial S_i(x_t)}{\partial x_t}] \quad (1.19)$$

通过上述两个步骤，可以求得机器人位姿的最优估计，使用金字塔存储的方法以防止陷入局部最优，将算法建立的地图按照栅格精度从低到高存储在金字塔的每个图层中。算法采用逐层搜索的方法，优先从低精度的地图展开所搜并获得最优解，接着将这个解用作更高精度地图的初始估计。

1.1.2 Gmapping 算法

Gmapping 算法基于粒子滤波，每个粒子都代表机器人此刻可能的位置，并存储了一幅地图。在数百平米的中小环境下建图时，Gmapping 算法能够很好地兼顾建图精度与计算量。Gmapping 需要里程计信息提供位姿初值并根据激光扫描的信息调整位姿。

(1) 基于 Rao-Blackwellized 粒子滤波的 SLAM 算法

RBPF SLAM 算法的目的是求取机器人各时刻的位姿 x_{t_r} 和全局地图 m 。基于粒子滤波的算法难以直接求解这两个变量的联合概率密度，因此往往是先求解一个变量并在此基础上求解另一个变量。实际算法先计算出机器人当前时刻的位姿，然后根据激光雷达的信息更新地图，后验概率可以分解为两个部分：

$$p(x_{t:t}, m | z_{t:t}, u_{t:t-1}) = p(x_{t:t} | z_{t:t}, u_{t:t-1}) p(m | x_{t:t}, z_{t:t}) \quad (1.20)$$

其中, $p(x_{t:t}, m | z_{t:t}, u_{t:t-1})$ 表示机器人状态的后验概率, $z_{t:t}$ 表示激光雷达 1 到 t 时刻的观测信息, $u_{t:t-1}$ 表示机器人 1 到 $t-1$ 时刻的控制信息, $x_{t:t}$ 表示机器人 1 到 t 时刻的位姿。若 $x_{t:t}$ 和 $z_{t:t}$ 已知, 则地图的后验概率 $p(m | x_{t:t}, z_{t:t})$ 可以直接求出。由 $p(x_{t:t} | z_{t:t}, u_{t:t-1})$ 可知, 机器人的各时刻的位姿可以通过激光雷达的观测信息以及机器人的运动控制信息计算得到。

SLAM 算法的步骤如下:

1□ 初始化: 初始时刻, 算法根据先验概率初始化粒子集 $\{X_t^{(i)}\}_{i=0,1,\dots,N}$, 粒子权重都为 $\omega_0^{(i)} = 1/N$ 。

2□ 重要性采样: 当前时刻的粒子集可由上一时刻的粒子集计算求得, 一般用机器人运动模型作为密度函数进行采样即 $\pi(x_t^{(i)} | x_{t-1}^{(i)}, z_t, u_{t-1}) = p(x_t^{(i)} | x_{t-1}^{(i)}, u_{t-1})$ 。

如图 1.10 所示, $f(x)$ 为目标分布, 包含里程计以及激光雷达的观测信息。激光雷达信息是离散的点, 因此无法对雷达信息进行建模, 也因此无法直接从 $f(x)$ 中采样。因此使用提议分布代替目标分布进行采样获取机器人下一个时刻的位姿信息, 提议分布和目标分布的差异性由粒子的权重反映。

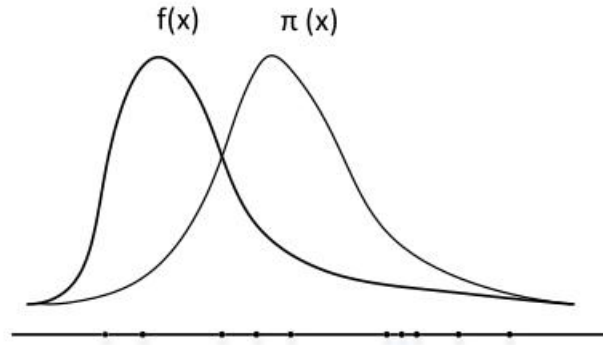


图 1.10 真实分布与建议分布

Fig 1.10 Actual distribution and proposed distribution

3□ 计算粒子权重: $\omega_t^{(i)}$ 用来表示重要性权重, 反应提议分布和目标分布的相似性, 大小两者的比值。 $\omega_t^{(i)}$ 取值越大, 则说明估计的位姿越接近真实值。

$$\begin{aligned} \omega_t^{(i)} &\propto \omega_{t-1}^{(i)} \frac{p(x_t^{(i)} | x_{t-1}^{(i)}, u_{t-1}) p(z_t | m_{t-1}^{(i)}, x_t^{(i)})}{\pi(x_t^{(i)} | x_{t-1}^{(i)}, z_t, u_{t-1})} \\ &= \omega_{t-1}^{(i)} p(z_t | m_{t-1}^{(i)}, x_t^{(i)}) \end{aligned} \quad (1.21)$$

4□ 重采样: 机器人在建图过程中, 有时受到环境特点以及传感器噪声的影响, 正确的粒子个数较少而无用粒子占用较多的资源, 导致机器人无法准确估计实时位姿。为了缓解这种现象, 提出重采样方法。当粒子状态较为分散时, 算法复制高权重粒子而舍弃低权重粒子, 这种方法既优化了粒子的分布, 同时也提高

了算法的计算效率。

5□ 更新地图：根据当前时刻机器人的位姿估计结合该时刻的观测信息 z_t 来更新地图。

(2) 改进 Rao-Blackwellized 滤波的 SLAM 算法

1□ 改进提议分布函数：

基于 RBPF 的 SLAM 算法使用机器人里程计模型作为建议分布函数，然而里程计模型易受噪声以及实际环境影响，具有较大的误差。相比之下雷达的观测数据更加准确，如图 1.11 所示，观测信息的似然分布相比运动模型曲线更加陡峭，其对机器人的位姿估计具有更高的置信度。因此在提议分布中融入观测的信息会使得位姿估计具有更高的精度。

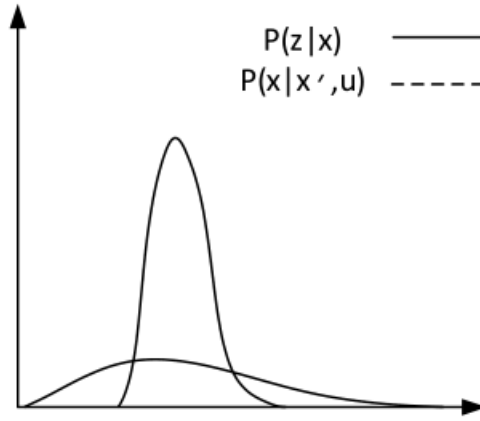


图 1.11 激光雷达模型与运动模型概率值

Fig 1.11 Probability values of lidar model and motion model

Giorgio Grisett 等人提出了将机器人最近的观测信息 z_t 融入体提议分布中的方法^[29]，如下式所示：

$$p(x_t | m_{t-1}^{(i)}, x_{t-1}^{(i)}, z_t, u_{t-1}) = \frac{p(z_t | m_{t-1}^{(i)}, x_t) p(x_t | x_{t-1}^{(i)}, u_{t-1})}{p(z_t | m_{t-1}^{(i)}, x_{t-1}^{(i)}, u_{t-1})} \quad (1.21)$$

得到最优建议分布函数之后，可以计算得到新的权值：

$$\begin{aligned} \omega_t^{(i)} &= \omega_{t-1}^{(i)} \frac{p(z_t | m_{t-1}^{(i)}, x_t^{(i)}) p(x_t^{(i)} | x_{t-1}^{(i)}, u_{t-1})}{p(x_t | m_{t-1}^{(i)}, x_{t-1}^{(i)}, z_t, u_{t-1})} \\ &= \omega_{t-1}^{(i)} p(z_t | m_{t-1}^{(i)}, x_{t-1}^{(i)}, u_{t-1}) \\ &= \omega_{t-1}^{(i)} \int p(z_t | m_{t-1}^{(i)}, x') p(x' | x_{t-1}^{(i)}, u_{t-1}) dx' \end{aligned} \quad (1.22)$$

上式的积分难以直接求解，因此实际计算时，在每个粒子的最大似然处采样 K 个点即 $x_j \in (x_1, x_2, \dots, x_K)$ ，利用这 K 个点可以求解出高斯模型的方差和平均值从而模

拟出提议分布函数。

$$\eta_t^{(i)} = \sum_{j=1}^k p(z_t | m_{t-1}^{(i)}, x_j) p(x_j | x_{t-1}^{(i)}, u_{t-1}) \quad (1.23)$$

$$\mu_t^{(i)} = \frac{1}{\eta_t^{(i)}} \sum_{j=1}^k x_j p(z_t | m_{t-1}^{(i)}, x_j) p(x_j | x_{t-1}^{(i)}, u_{t-1}) \quad (1.24)$$

$$\sum_i^i = \frac{1}{\eta_t^{(i)}} \sum_{j=1}^k (x_j - \mu_t^{(i)})(x_j - \mu_t^{(i)})^T p(z_t | m_{t-1}^{(i)}, x_j) p(x_j | x_{t-1}^{(i)}, u_{t-1}) \quad (1.25)$$

此时的权值计算公式是：

$$\begin{aligned} \omega_t^{(i)} &= \omega_{t-1}^{(i)} p(z_t | m_{t-1}^{(i)}, x_{t-1}^{(i)}, u_{t-1}) \\ &= \omega_{t-1}^{(i)} \int p(z_t | m_{t-1}^{(i)}, x') p(x' | x_{t-1}^{(i)}, u_{t-1}) dx' \\ &= \omega_{t-1}^{(i)} \sum_{j=1}^k p(z_t | m_{t-1}^{(i)}, x_j) p(x_j | x_{t-1}^{(i)}, u_{t-1}) \\ &= \omega_{t-1}^{(i)} \eta_t^{(i)} \end{aligned} \quad (1.26)$$

得到了改进后的提议分布函数和权值更新公式，该公式融合了里程计数据和最新时刻的观测数据，可以更加准确地采样粒子。

2□ 自适应重采样：

Gmapping 算法使用自适应重采样的方法防止粒子过度发散，这种方法有效了减少了重采样的次数^[30,31,32]。当计算出的有效粒子数 N_{eff} 小于设定值时重采样，否则直接计算位姿，避免粒子贫化。 N_{eff} 的计算公式为：

$$N_{eff} = \frac{1}{\sum_{i=1}^n (\omega_t^{(i)})^2} \quad (1.27)$$

阈值通常设置为：

$$N_{th} = \frac{N}{2} \quad (1.28)$$

将有效粒子数 N_{eff} 与设定阈值 N_{th} 进行比较，当 $N_{eff} < N_{th}$ 则执行重采样，反之则跳过重采样。

Gmapping 算法改进了提议分布函数和重采样方法，提高采样精度的同时减少了计算量。

1.4 SLAM 建图仿真

本文的研究面向医疗方舱环境,选用建图算法时应考虑到算法对硬件的要求和建图精度两个问题。Cartographer 算法有闭环检测模块，建图精度高，但是对主机性能要求相对较高。Hector SLAM 算法主要应用于地面不平坦的环境，建图

精度相对较低且对激光雷达要求较高。综合考虑，本文使用 Gmapping 算法进行建图。

1.4.1 仿真环境的搭建

Gazebo 是一个开源与 ROS 兼容性良好的三维物理学仿真平台。在 Gazebo 中使用者能够搭建各种虚拟的仿真环境以及自定义机器人模型，在忽略机器人打滑以及实际环境中的干扰问题时，Gazebo 的仿真结果具有较高的可信度。因此 Gazebo 被广泛应用于算法测试、机器人设计以及 AI 训练等各个方面。本节使用 Gazebo 平台搭建第一章医疗方舱的仿真环境，Gazebo 应用界面如图 1.12 所示。

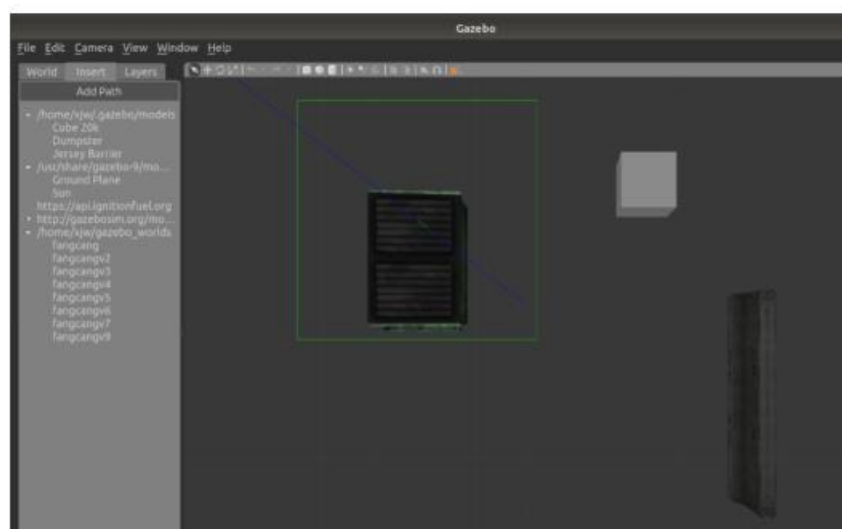


图 1.12 Gazebo 仿真界面

Fig 1.12 Gazebo simulation interface

界面中间的部分可以直观地显示搭建地仿真环境。左侧的状态栏包括三个子栏：World 栏可以查看仿真环境的组成部件并设置相应的状态信息；Insert 栏连接了仿真环境的库，使用者搭建环境时可以直接从中调用想要的物体模型；Layers 栏则让使用者可以搭建多层的复杂环境模型。

本文搭建的方舱环境属于典型的结构化环境，需要知道各个墙面的精确位置，因此直接调用库里的模型不能够达到目标。点击上方“Edit”栏选择“Building Editor”进入环境编辑界面，如图 1.13 所示。

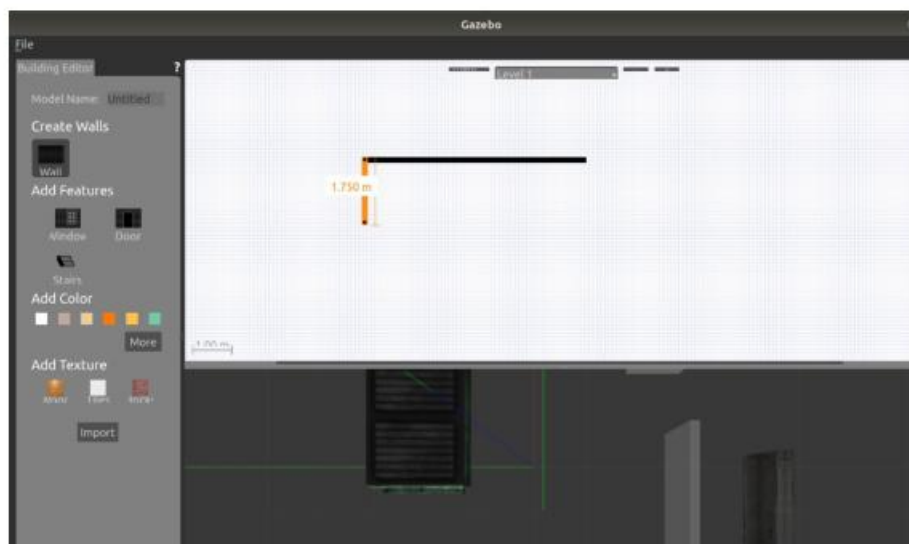


图 1.13 Gazebo 环境编辑界面

Fig 1.13 Gazebo environment editing interface

选择“Create Walls”即可搭建方舱环境，搭建完成之后将模型保存到合适的路径。再次打开 Gazebo，在“Insert”栏即可调出该模型。本文搭建的方舱环境如图 1.14 所示。每个单独的方舱设置出入口，地图面积 31m*23m，方舱外设置储物柜以增加环境特征。

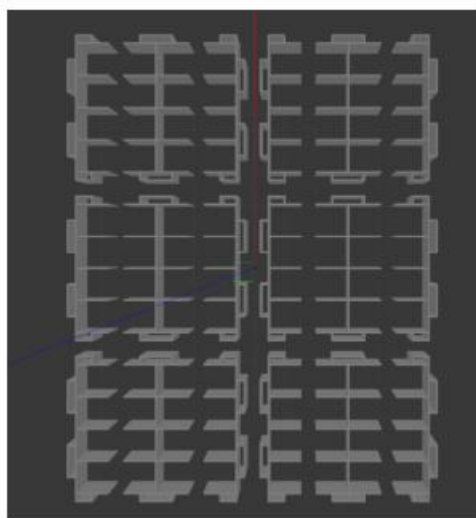


图 1.14 方舱仿真环境

Fig 1.14 Square Cabin Simulation Environment

1.4.2 建图质量评价方法

本文拟研究方舱环境下，Gmapping 关键参数对建图精度的影响，提高建图精度是根本目的。但是由于地图的复杂性，无论是在仿真环境还是真实环境都不

能直接从建立的地图本身进行评价。现在对建图质量多是从定位的角度进行评价的，通过获取机器人在仿真环境中的真值位置 $P_{truth} = (x_t, y_t)$ 和算法给出的位置 $P_{gmap} = (x'_t, y'_t)$ ，计算两者距离的均方根误差即可。计算公式如下：

$$d = \sqrt{\frac{\sum_{k=1}^N ((x_k - x'_k)^2 + (y_k - y'_k)^2)}{N}} \quad (1.29)$$

其中 N 代表建图过程中累计采样的次数， d 代表建图过程中机器人的平均定位误差，可直接用于量化表示建图精度。

机器人在仿真环境下移动时，实时位姿对应于/map 和/base_link 坐标系的转换关系，在终端界面输入“roslaunch tf_echo /map /base_link”即可输出算法给出的机器人位姿。而机器人的真值位姿可以通过编写节点订阅

“/gazebo/model_states”话题获取。ROS 环境下有计算定位均方根的工具 evo，但是 evo 常用于数据集的评测，并且 evo 对两组数据时间戳对齐要求较高，一般在 0.01s 内。本文使用 tf 包输出坐标信息，发布频率为 1hz，所以需要将数据导出用 Matlab 进行处理。导出的数据如图 1.15 所示。

文件(F)	编辑(E)	格式(O)	查看(V)	帮助	文件(F)	编辑(E)	格式(O)	查看(V)	帮助
413.851	0	0.002			404.526	-0.000242	0.001382		
414.851	0.001	0.002			405.128	-0.000244	0.001395		
415.851	0.003	0			405.755	-0.000247	0.001409		
416.851	0	0			406.357	-0.000249	0.001423		
417.851	0.001	0			406.947	-0.000252	0.001436		
418.851	0	0			407.562	-0.000254	0.00145		
419.851	0	0			408.172	-0.000257	0.001464		
420.851	0.003	0			408.803	-0.00026	0.001479		
421.851	0.001	0			409.425	-0.000262	0.001493		
422.851	0	0.002			410.051	-0.000265	0.001507		
423.851	0	-0.002			410.705	-0.000267	0.001522		
424.851	0	0			411.338	-0.00027	0.001536		
425.851	-0.003	0			411.97	-0.000273	0.001551		
426.851	0.028	0.001			412.635	-0.000276	0.001566		
427.851	0.206	0.007			413.23	-0.000278	0.00158		
428.851	0.426	0.011			413.902	-0.000281	0.001595		
429.851	0.639	0.016			414.523	-0.000284	0.001609		
430.851	0.869	0.027			415.165	-0.000286	0.001624		
431.851	1.081	0.034			415.795	-0.000289	0.001638		
432.851	1.293	0.044			416.418	-0.000292	0.001652		
433.851	1.531	0.053			417.081	-0.000294	0.001668		

图 1.15 真实数据与计算位置数据

Fig 1.15 Real location and calculated location data

处理数据的第一步是将数据按照时间戳进行对齐。算法给出数据的频率较低，所以对计算的数据，需要从真值数据里找出与之时间距离最近的数据进行匹配。如图 1.17 中红色箭头所示，整个过程中机器人基本是处于匀速运动的状态，所以在 411.851 时间点，机器人的真值位置可以通过 411.23 和 413.902 两个时刻的位置插值获得。匹配后的数据可以进一步绘制成轨迹，直观地展现定位误差。使用 Gmapping 算法在方舱环境下单通道建图测试，平均定位误差为 0.373m，真值

与计算轨迹如图 1.16 所示。

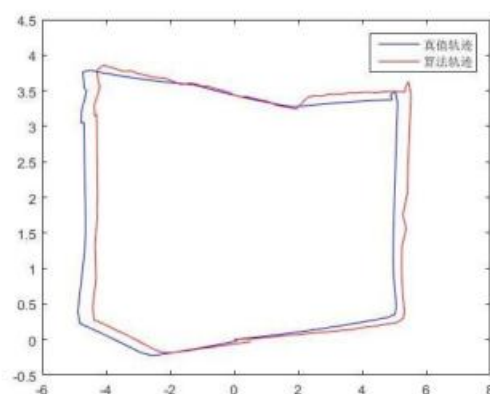


图 1.16 算法和真值轨迹

Fig 1.16 Algorithm and truth locus

1.4.3 建图关键参数的选择

使用默认的参数建图往往会导致建图精度变差，所以调用 Gmapping 算法需要对算法以及机器人的参数进行配置。国内外有一些学者对建图的参数进行了研究^[9,33,34,35]，但是这些研究没有分析环境特点给机器人建图带来的不利影响，且只靠肉眼对建图效果进行评价，其对参数的研究有失一般性。

医疗方舱是一个典型的高相似度多闭环环境，其多闭环的特点使得机器人在内部建图时需要频繁地转弯，而转弯正是机器人建图最容易出错的时候。针对机器人频繁转弯导致建图精度变差的问题，有两个参数的选值影响比较大：

Gmapping 算法的粒子数和机器人移动角速度。

一般来说粒子数直接取决于环境面积，在一定范围内，增加粒子数容易实现闭环，减少机器人移动过程中地累积误差，从而避免建图出现大规模错位的情况。这主要是由于两点原因：首先，增加粒子数使得粒子可能出现的位姿增多，粒子群收敛到错误位姿的可能性减小；其次，增加粒子数使得粒子群重采样的次数显著降低，降低了粒子贫化的可能性。在搭建的环境里使用 Turtlebot3 进行仿真，建图效果随粒子数变化的情况如图 1.17 所示。可以发现当粒子数较少时，第三行方舱的地图发生了严重的错位，而随着粒子数的增多，情况逐渐好转。当粒子数取到 80 个时，算法运算量过大导致运行速度明显降低。综合考虑建图精度与算法计算量，在本文搭建的环境中，粒子数取 60 最合适。

Gmapping 算法在中小环境的建图效果最佳，一般在 1000 m^2 以内。粒子数可以根据环境的面积设定初值并根据建图效果调整，当环境面积在 100 m^2 左右时可以设定粒子数为 30。如果建立的地图有明显的交叠错位问题时，则适量增加粒子数直至出错情况减少。

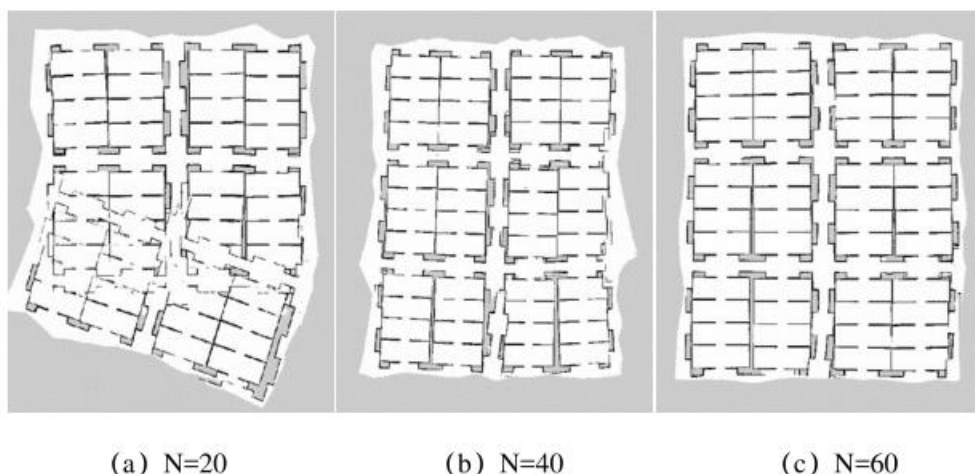


图 1.17 不同粒子数建图效果

Fig 1.17 The effect of different particle numbers

除了建图算法本身的参数外，机器人本身的运动参数也是影响建图质量的重要因素。机器人在直线运动过程中，建图精度较高同时稳定性也较好。但是机器人在旋转时，往往会出现很多的问题，包括激光点云的畸变，机器人的打滑等，这些问题会使得建图算法容易帧间匹配失败以及粒子频繁地重采样，从而导致建图精度变差，重采样如图 1.18 所示。使用者在操作机器人时，经常会重视线速度而忽略角速度的影响。以 Turtlebot3 机器人为例，该机器人建图时默认的线速度是 0.22m/s，角速度是 2.84rad/s，机器人在过快的角速度下旋转很难计算出位姿变换的信息。另一方面，如果将角速度设置地过小，机器人在方舱内移动建图的时间会明显增加。

```

Laser Pose= 0.112023 3.18553 0.0315394
n_count 147
Average Scan Matching Score=253.976
neff= 50.9938
Registering Scans:Done
update frame 426
update id=0.131913 ad=0.00338807
Laser Pose= -0.0198235 3.18134 0.0321826
n_count 148
Average Scan Matching Score=235.013
neff= 36.3361
Registering Scans:Done
update frame 429
update id=0.131943 ad=0.0021522
Laser Pose= -0.151694 3.17699 0.0332741
n_count 149
Average Scan Matching Score=259.797
neff= 27.7857
*****RESAMPLE*****
Deleting Nodes: 0 3 9 13 17 21 25 27 30 33 36 37 40 44 46 49 52 54 57 Done
Deleting old particles...Done
Copying Particles and Registering scans... Done
update frame 432
update id=0.131825 ad=0.00483602

```

图 1.18 参数设置不当导致重采样

Fig 1.18 Resampling caused by improper parameter setting

为了研究角速度对建图的影响，在单个方舱内进行建图仿真。机器人经过每个隔间时，原地旋转一圈，然后直行到下一个隔间直至走完整个方舱。建图时间以及建图过程中的定位误差随角速度的变化情况如表 1.1 以及图 1.19 所示。综合

考虑建图精度与时间的影响，角速度取 0.4-0.45rad/s 最合适。

表 1.1 建图精度随角速度变化关系

Table 1.1 The relationship between the accuracy of mapping and the angular velocity

角速度 rad/s	建图时间 s	定位平均误差 m	定位误差标准差 m
0.2	480	0.1534	0.0906
0.25	426	0.1698	0.0798
0.3	393	0.1479	0.1187
0.35	361	0.1591	0.0692
0.4	341	0.1222	0.0689
0.45	326	0.1191	0.0802
0.5	314	0.1316	0.1021
0.55	303	0.1472	0.0779
0.6	294	0.1581	0.0933
0.65	287	0.1744	0.0762

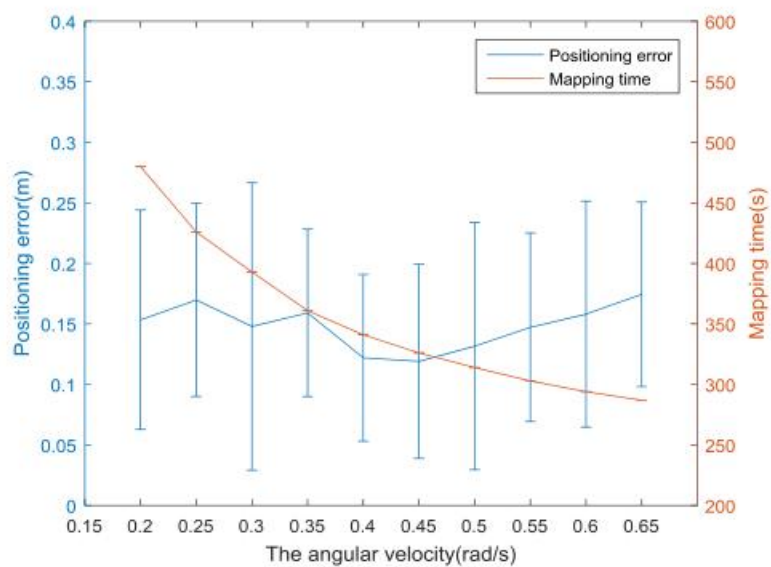


图 1.19 建图时间及定位误差随角速度变化关系

Fig 1.19 Relation of mapping time and positioning error with angular velocity

1.5 实验与分析

为了验证参数的选择对建图精度的影响，用搭建的机器人在逸夫楼 1107 实验室进行建图实验。1107 实验室是典型的工作间环境，可以用来模拟方舱环境，机器人实物以及实验环境分别如图 1.20 和 1.21 所示。



图 1.20 移动机器人实物图

Fig 1.20 Real robot picture



图 1.21 1107 实验室环境

Fig 1.21 1107 Laboratory environment

首先使用 Turtlebot 软件包配置文件的默认参数进行建图，其中粒子数为 8，机器人角速度为 1rad/s 。实际建图效果如图 1.22 所示。图片的左下角是实验室入口处，机器人从下往上依次遍历三个通道建立环境地图，可以发现所建立的环境地图存在两个问题：单个通道所呈现的效果较为弯曲，这是因为默认的角速度过大，机器人转弯时，帧间匹配引入了过多的误差；另一方面，建立的地图上面两行通道发生了错位重叠的问题，这主要是由于粒子数设置不当粒子重采样造成的。

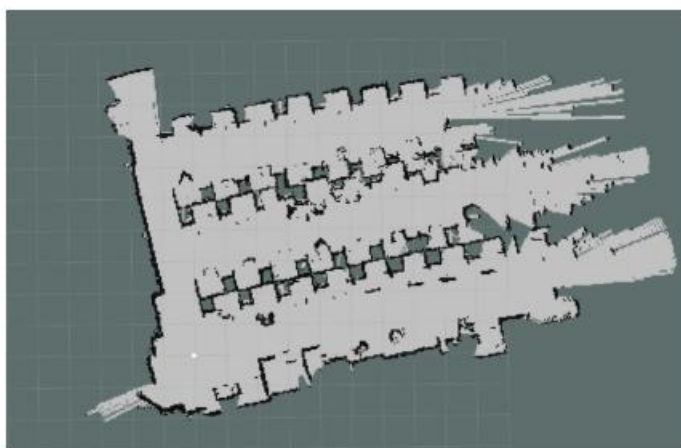


图 1.22 Gmapping 默认参数建图

Fig 1.22 Gmapping for default parameters

根据上文总结出的参数规律对默认参数进行调整，粒子数调整为 30，线性角速度调整为 0.45rad/s ，建立的环境地图如图 1.23 所示。可以发现建立的地图细节上较默认参数建立的图更加完善，并且没有出现通道弯曲以及交叠的问题。

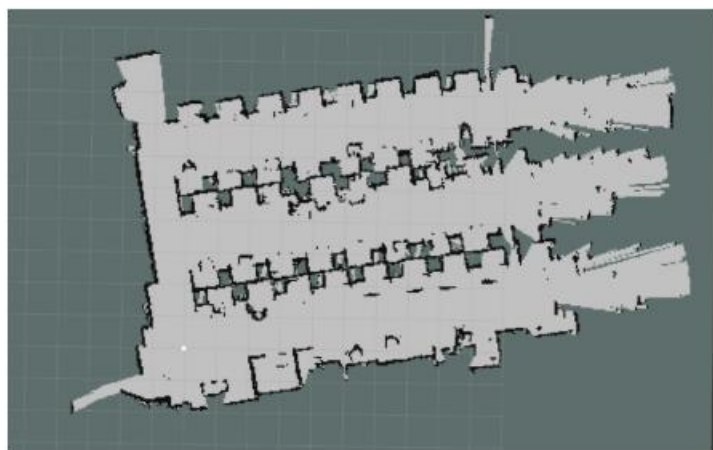


图 1.23 Gmapping 调整参数建图

Fig 1.23 Gmapping adjusted parameter mapping